| | |
|---|---|
| **Name: Alegre, Jhon Isaac S.** | **Date Performed: 12/11/2025** |
| **Course/Section:CPE212-31S1** | **Date Submitted: 12/11/2025** |
| **Instructor:** | **Semester and SY: 2025-2026** |

**Activity 2: SSH Key-Based Authentication and Setting up Git**

1. **Objectives:**

1.1 Configure remote and local machine to connect via SSH using a KEY instead of using a password
1.2 Create a public key and private key
1.3 Verify connectivity
1.4 Setup Git Repository using local and remote repositories
1.5 Configure and Run ad hoc commands from local machine to remote servers

**Part 1: Discussion**

It is assumed that you are already done with the last Activity (**Activity 1: Configure Network using Virtual Machines).** *Provide screenshots for each task*.

It is also assumed that you have VMs running that you can SSH but requires a password. Our goal is to remotely login through SSH using a key without using a password. In this activity, we create a public and a private key. The private key resides in the local machine while the public key will be pushed to remote machines. Thus, instead of using a password, the local machine can connect automatically using SSH through an authorized key.

**What Is ssh-keygen?**

Ssh-keygen is a tool for creating new authentication key pairs for SSH. Such key pairs are used for automating logins, single sign-on, and for authenticating hosts.

**SSH Keys and Public Key Authentication**

The SSH protocol uses public key cryptography for authenticating hosts and users. The authentication keys, called SSH keys, are created using the keygen program.

SSH introduced public key authentication as a more secure alternative to the older .rhosts authentication. It improved security by avoiding the need to have password stored in files and eliminated the possibility of a compromised server stealing the user's password.

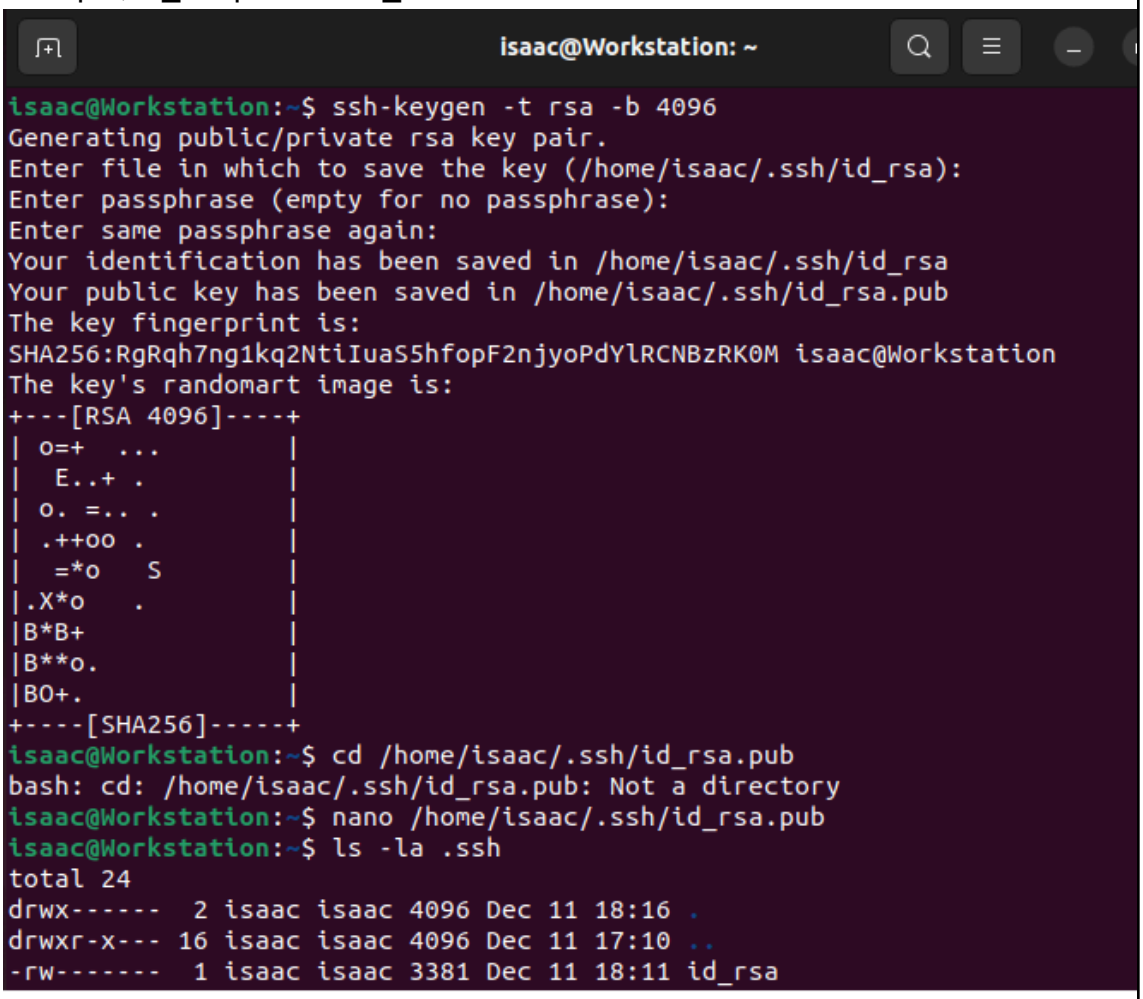
However, SSH keys are authentication credentials just like passwords. Thus, they must be managed somewhat analogously to usernames and passwords. They should have a proper termination process so that keys are removed when no longer needed.

**Task 1: Create an SSH Key Pair for User Authentication**

1. The simplest way to generate a key pair is to run *ssh-keygen* without arguments. In this case, it will prompt for the file in which to store keys. First,

the tool asked where to save the file. SSH keys for user authentication are usually stored in the users .ssh directory under the home directory. However, in enterprise environments, the location is often different. The default key file name depends on the algorithm, in this case *id_rsa* when using the default RSA algorithm. It could also be, for example, *id_dsa* or *id_ecdsa*.

2. Issue the command *ssh-keygen -t rsa -b 4096.* The algorithm is selected using the -t option and key size using the -b option.
3. When asked for a passphrase, just press enter. The passphrase is used for encrypting the key, so that it cannot be used even if someone obtains the private key file. The passphrase should be cryptographically strong.
4. Verify that you have created the key by issuing the command *ls -la .ssh.* The command should show the .ssh directory containing a pair of keys. For example, id_rsa.pub and id_rsa.




**Task 2: Copying the Public Key to the remote servers**

1. To use public key authentication, the public key must be copied to a server and installed in an *authorized_keys* file. This can be conveniently done using the *ssh-copy-id* tool.

2. Issue the command similar to this: *ssh-copy-id -i ~/.ssh/id_rsa user@host*
3. Once the public key has been configured on the server, the server will allow any connecting user that has the private key to log in. During the login process, the client proves possession of the private key by digitally signing the key exchange.
4. On the local machine, verify that you can SSH with Server 1 and Server 2. What did you notice? Did the connection ask for a password? If not, why?

```
isaac@Workstation:~$ ssh isaac@192.168.56.104
Welcome to Ubuntu 22.04.2 LTS (GNU/Linux 6.8.0-87-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

 * Introducing Expanded Security Maintenance for Applications.
   Receive updates to over 25,000 software packages with your
   Ubuntu Pro subscription. Free for personal use.

     https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Thu Dec 11 17:31:25 2025 from 192.168.56.102
isaac@ManagerNode:~$
```

**I noticed that i dont need to type password, so the connection did not ask any password after typing the command.**

**Reflections:**

Answer the following:

1. How will you describe the ssh-program? What does it do?
   - **SSH (Secure Shell) is a tool that lets you securely connect to and manage remote servers over an encrypted channel. It protects your communication from eavesdropping and allows you to run commands, transfer files, or tunnel connections safely**
2. How do you know that you already installed the public key to the remote servers?
   - **You know the public key is installed if you can log in to the remote server without being asked for a password, meaning the server recognizes and accepts your key.**

**Part 2: Discussion**

*Provide screenshots for each task*.

It is assumed that you are done with the last activity (**Activity 2: SSH Key-Based Authentication**).

**Set up Git**

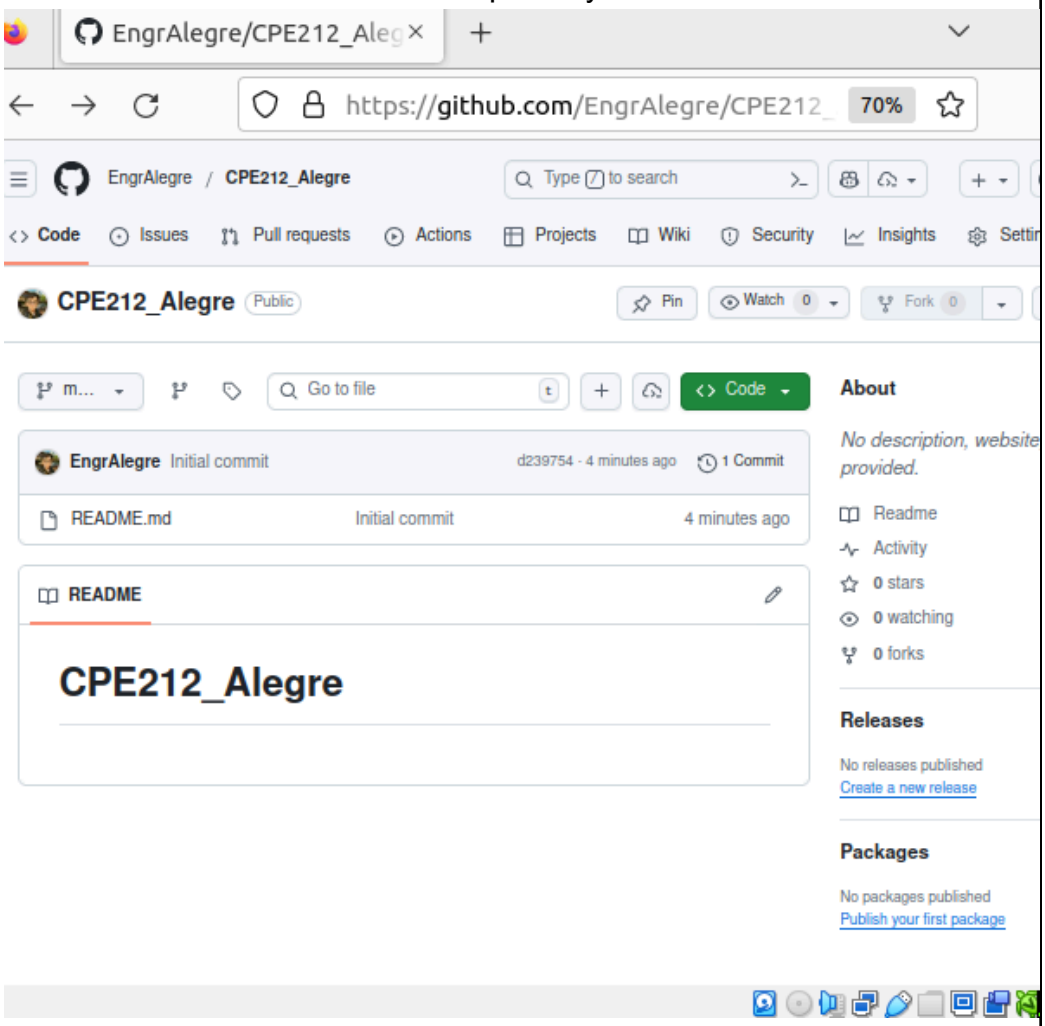
At the heart of GitHub is an open-source version control system (VCS) called Git. Git is responsible for everything GitHub-related that happens locally on your computer. To use Git on the command line, you'll need to download, install, and configure Git on your computer. You can also install GitHub CLI to use GitHub from the command line. If you don't need to work with files locally, GitHub lets you complete many Git-related actions directly in the browser, including:

- Creating a repository
- Forking a repository
- Managing files
- Being social

**Task 3: Set up the Git Repository**

1. On the local machine, verify the version of your git using the command *which git.* If a directory of git is displayed, then you don't need to install git. Otherwise, to install git, use the following command: *sudo apt install git*
2. After the installation, issue the command *which git* again. The directory of git is usually installed in this location: *user/bin/git.*
3. The version of git installed in your device is the latest. Try issuing the command *git --version* to know the version installed.
4. Using the browser in the local machine, go to www.github.com.
5. Sign up in case you don't have an account yet. Otherwise, login to your GitHub account.

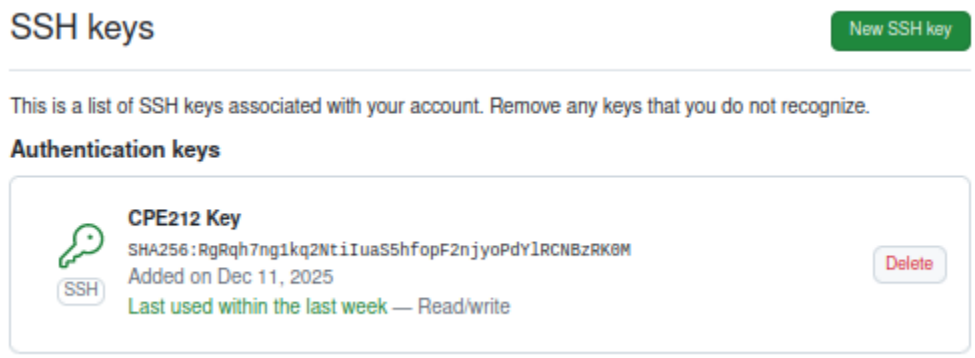
a. Create a new repository and name it as CPE232_yourname. Check Add a README file and click Create repository.



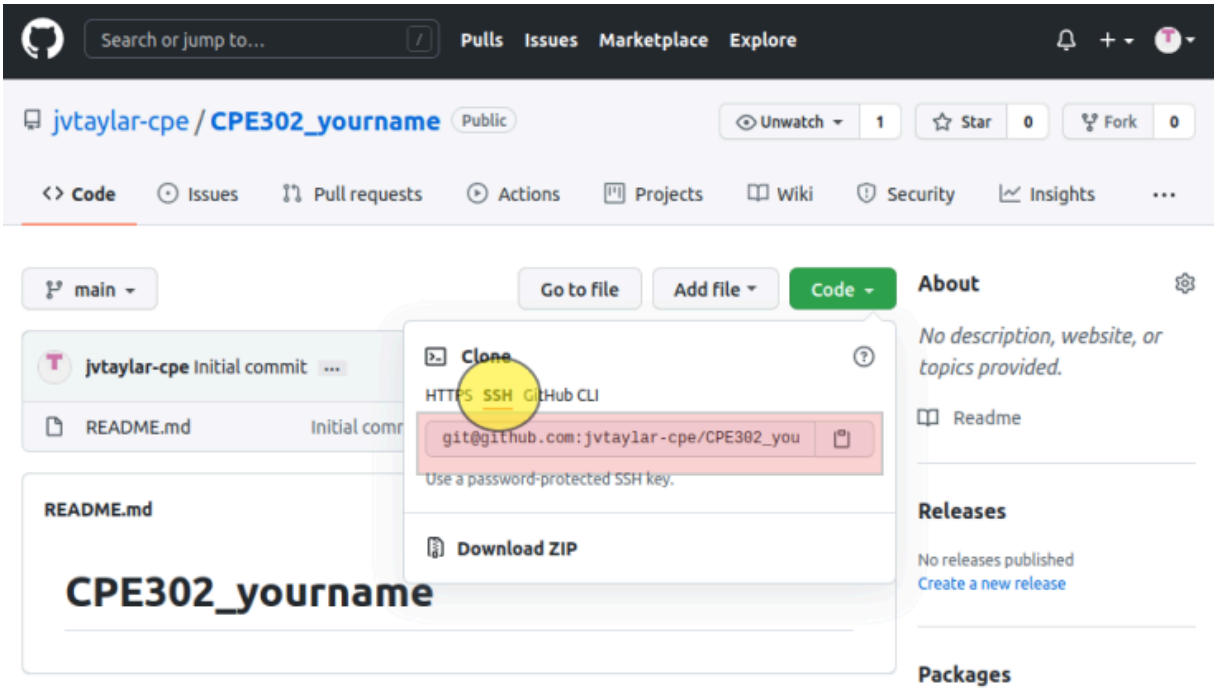

b. Create a new SSH key on GitHub. Go your profile's setting and click SSH and GPG keys. If there is an existing key, make sure to delete it. To create a new SSH keys, click New SSH Key. Write CPE232 key as the title of the key.

c. On the local machine's terminal, issue the command cat .ssh/id_rsa.pub and copy the public key. Paste it on the GitHub key and press Add SSH

key.




d. Clone the repository that you created. In doing this, you need to get the link from GitHub. Browse to your repository as shown below. Click on the Code drop down menu. Select SSH and copy the link.




e. Issue the command git clone followed by the copied link. For example, *git clone* *[git@github.com:jvtaylar-cpe/CPE232_yourname.git](git@github.com:jvtaylar-cpe/CPE232_yourname.git)*. When prompted to continue connecting, type yes and press enter.

f. To verify that you have cloned the GitHub repository, issue the command *ls*. Observe that you have the CPE232_yourname in the list of your directories. Use CD command to go to that directory and LS

command to see the file README.md.

```
isaac@Workstation:~$ ls
CPE212          Desktop    Downloads  Pictures  snap       Videos
CPE212_Alegre   Documents  Music      Public    Templates
```

g. Use the following commands to personalize your git.
   - *git config --global user.name "Your Name"*
   - *git config --global user.email yourname@email.com*
   - Verify that you have personalized the config file using the command *cat ~/.gitconfig*

h. Edit the README.md file using nano command. Provide any information on the markdown file pertaining to the repository you created. Make sure to write out or save the file and exit.

i. Use the *git status* command to display the state of the working directory and the staging area. This command shows which changes have been staged, which haven't, and which files aren't being tracked by Git. Status output does not show any information regarding the committed project history. What is the result of issuing this command?

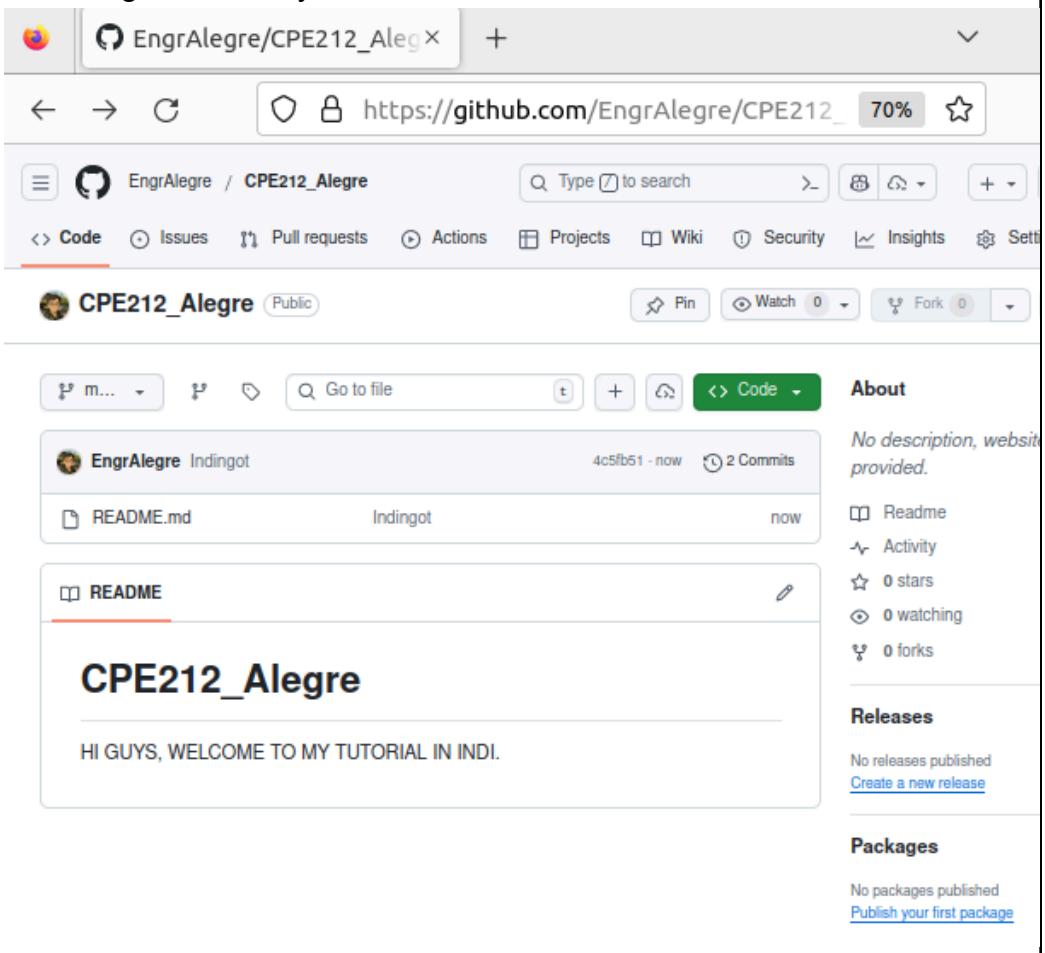
j. Use the command *git add README.md* to add the file into the staging area.

k. Use the *git commit -m "your message"* to create a snapshot of the staged changes along the timeline of the Git projects history. The use of this command is required to select the changes that will be staged for the next commit.

l. Use the command *git push <remote><branch>* to upload the local repository content to GitHub repository. Pushing means to transfer commits from the local repository to the remote repository. As an example, you may issue *git push origin main*.

```
isaac@Workstation:~/CPE212_Alegre$ nano README.md
isaac@Workstation:~/CPE212_Alegre$ git add .
isaac@Workstation:~/CPE212_Alegre$ git commit -m "Indingot"\
> ^C
isaac@Workstation:~/CPE212_Alegre$ git commit -m "Indingot"
[main 4c5fb51] Indingot
 1 file changed, 3 insertions(+), 1 deletion(-)
isaac@Workstation:~/CPE212_Alegre$ git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 301 bytes | 301.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:EngrAlegre/CPE212_Alegre.git
   d239754..4c5fb51  main -> main
isaac@Workstation:~/CPE212_Alegre$
```

m. On the GitHub repository, verify that the changes have been made to README.md by refreshing the page. Describe the README.md file. You can notice the how long was the last commit. It should be some minutes ago and the message you typed on the git commit command should be there. Also, the README.md file should have been edited according to the text you wrote.




**Reflections:**

Answer the following:

3. What sort of things have we so far done to the remote servers using ansible commands?

   -

4. How important is the inventory file?

   -

**Conclusions/Learnings:**

In conclusion I learned how to differentiate first the public and private key then use the public key to have a remote connection between two devices. In this activity or HOA, it enhanced my knowledge when it comes on ssh connection on how it is reliable and

also secured when it comes on remote connection. At the end I successfully completed the task given here on activity and helps me a lot on this topic to understand it more.